

Hybrid IAM Automation and Cloud Federation

Prepared by: Abdullah Mushtaq

Date: 5th June, 2026

Table of Contents

Executive Summary	3
Project Objective	3
Phase-by-Phase Execution	3
Business Value and Security Impact	4
Phase 1: The On-Prem Foundation	5
Phase 2: On-Premise Directory Setup and WinRM Bridge	8
Structuring Your Active Directory	8
Access Control & Security Auditing	10
Set Up the Automation Bridge (WinRM)	13
Configure VM Network Settings	13
Prepare the Python Environment (Host Machine)	14
Crafting the Provisioning Pipeline.....	14
Phase 3: Cryptographic Alignment and Agent Deployment	16
Phase 4: Cloud Identity Federation and Synchronization	18
Phase 5: De-provisioning Automation and Incident Response Readiness	24
The Python Offboarding Script	24

Executive Summary

Project Objective

The primary objective of this laboratory exercise was to design, deploy, and secure a hybrid Identity and Access Management (IAM) architecture. The project involved bridging an on-premise Active Directory (AD) environment with a cloud-based Identity Provider (Okta) to achieve seamless user federation. Furthermore, the lab focused on establishing an automation pipeline to programmatically enforce identity lifecycle management, specifically targeting zero-touch account de-provisioning to mitigate the risk of orphaned credentials.

Phase-by-Phase Execution

Phase 1: Infrastructure Provisioning and Network Configuration

- Deployed a Windows Server Virtual Machine to act as the primary Domain Controller.
- Architected a streamlined network configuration utilizing a Bridged Adapter, granting the virtual machine direct access to the local physical network. This approach seamlessly facilitated both local host-to-guest management and the required outbound internet connectivity for cloud communication through a single interface.

Phase 2: On-Premise Directory Setup and WinRM Bridge

- Configured Active Directory Domain Services (AD DS) and established a structured organizational unit (OU) hierarchy (Enterprise_Users).
- Provisioned initial directory objects (standard users and test administrative accounts).
- Enabled and secured Windows Remote Management (WinRM) to allow programmatic execution of PowerShell payloads from an external Python host.

Phase 3: Cryptographic Alignment and Agent Deployment

- Resolved native cryptographic protocol mismatches by forcing the Windows Server .NET framework to utilize modern TLS 1.2 standards.
- Bypassed strict Active Directory service account limitations to successfully deploy the Okta AD Agent, establishing a secure, outbound polling tunnel to the Okta cloud environment.

Phase 4: Cloud Identity Federation and Synchronization

- Configured directory integration rules within the Okta Admin Console, selectively mapping the targeted on-premise OU to the cloud directory.
- Implemented User Principal Name (UPN) mapping to ensure cloud identities maintained standard email-format routing.
- Executed a full directory import, successfully federating on-premise Active Directory objects into active, cloud-native Okta profiles.

Phase 5: De-Provisioning Automation and Incident Response Readiness

- Developed a custom Python application utilizing the winrm library to programmatically interface with the Domain Controller.
- Engineered a PowerShell payload that dynamically queries AD objects by UPN and executes immediate account suspension (Disable-ADAccount).

- Validated the lifecycle loop: the Python script successfully disabled an on-premise account, which subsequently triggered an automated access revocation in the Okta cloud during the next synchronization cycle.

Business Value and Security Impact

This architecture demonstrates a highly scalable approach to modern identity security. By replacing manual offboarding with an automated WinRM-to-PowerShell bridge, the environment is fully primed for advanced DevSecOps integration. This mechanism allows for direct communication with SIEM solutions and SOAR platforms. In the event of a critical security alert—such as anomalous lateral movement or compromised credentials—an automated webhook can immediately trigger the Python payload, ensuring the threat is isolated on-premise and access is revoked globally in the cloud with zero human latency.

Phase 1: The On-Prem Foundation

We are going to set up local Active Directory environment using virtualization. This will serve as our on-premise "source of truth" before we sync it to the cloud.

Step 1. Download the Prerequisites:

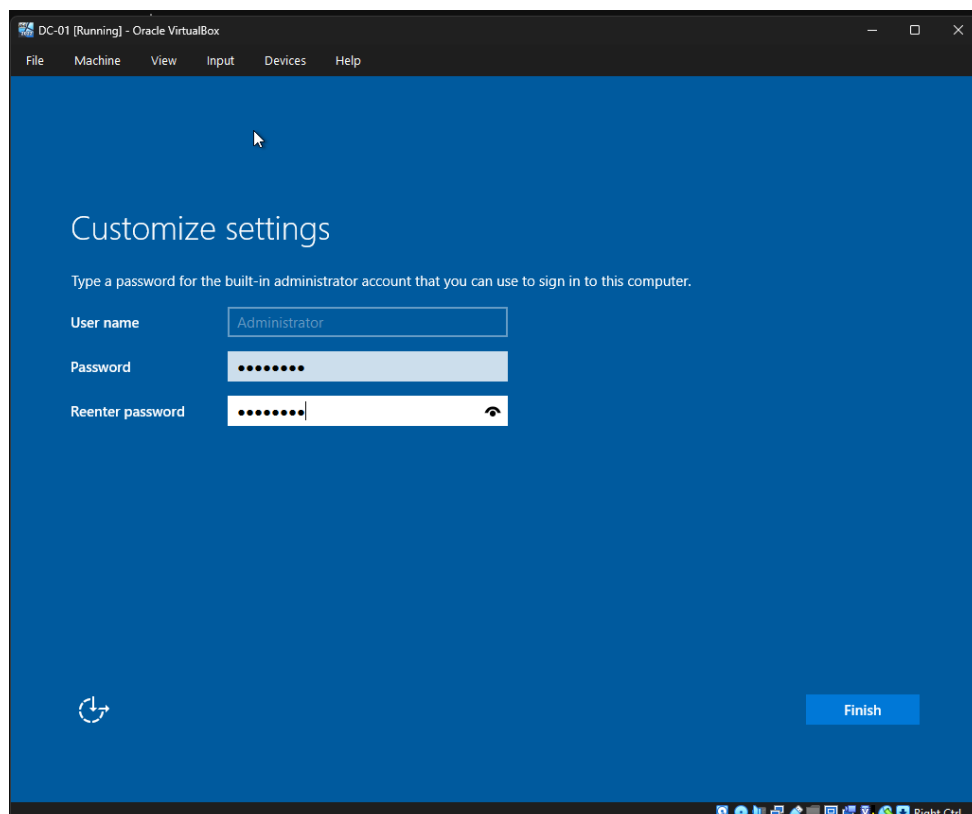
1. Download and install **Oracle VirtualBox** (free).
2. Download the **Windows Server 2022 Evaluation ISO** from the Microsoft Evaluation Center. You will need to fill out a brief form, but it is free and gives you 180 days of access.

Step 2. Create the Virtual Machine:

1. Open VirtualBox and create a new VM. Name it DC01 (Domain Controller 01).
2. Allocate at least 4GB of RAM and two CPU cores.
3. Create a virtual hard disk of about 50GB.
4. In the VM settings, go to Storage, click the empty optical drive, and mount the Windows Server 2022 ISO you downloaded.

Step 3. Install Windows Server:

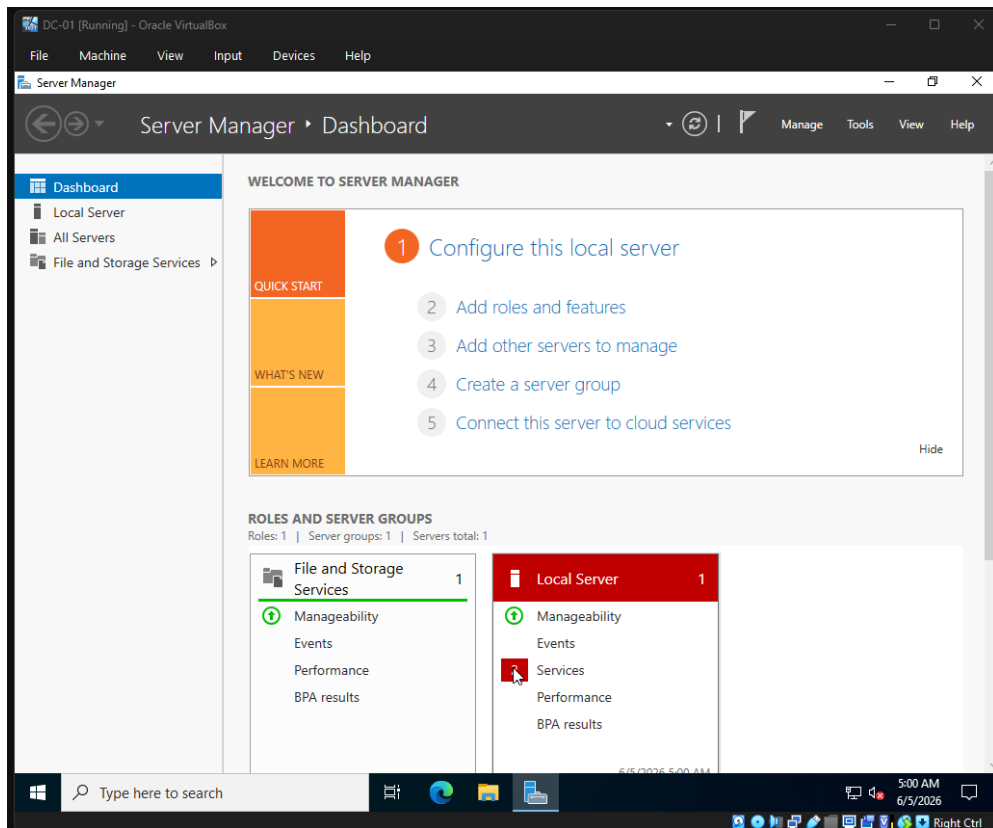
1. Start the VM. It will boot from the ISO.
2. Follow the standard Windows installation prompts.
3. **Crucial step:** When prompted for the installation type, choose **Windows Server 2022 Standard Evaluation (Desktop Experience)**. If you don't select "Desktop Experience," you will get a command-line-only interface.
4. Set a strong, memorable Administrator password.



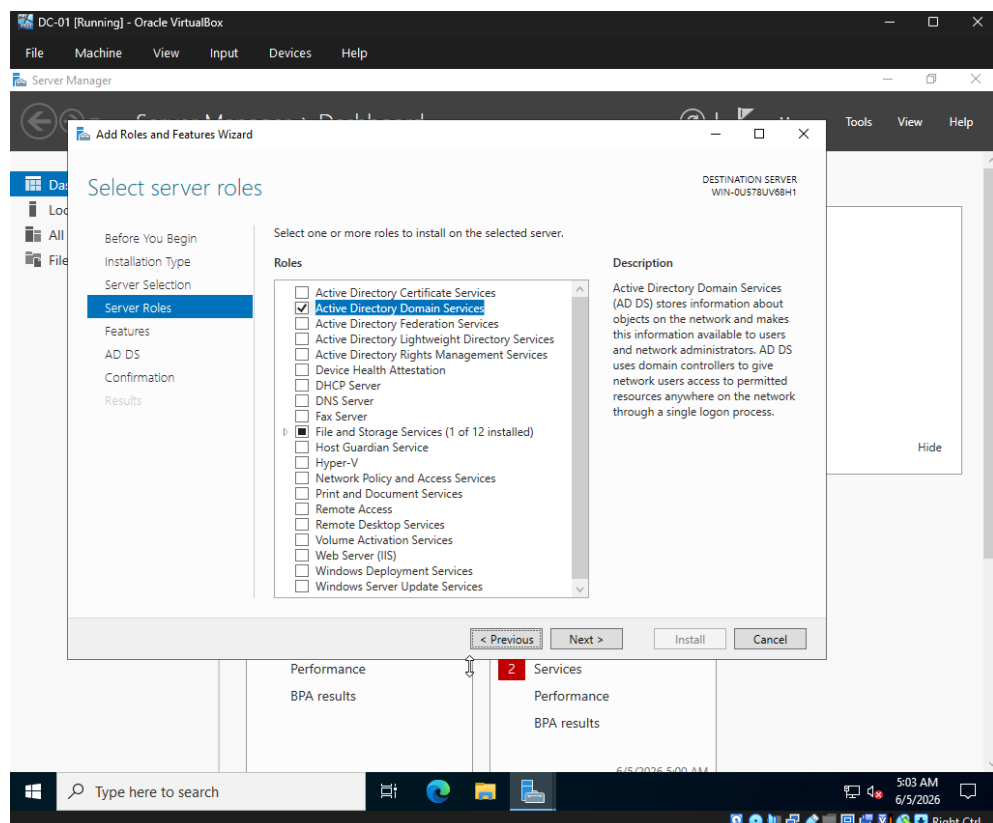
Step 4. Promote to a Domain Controller:

Once Windows Server is installed and you are logged in:

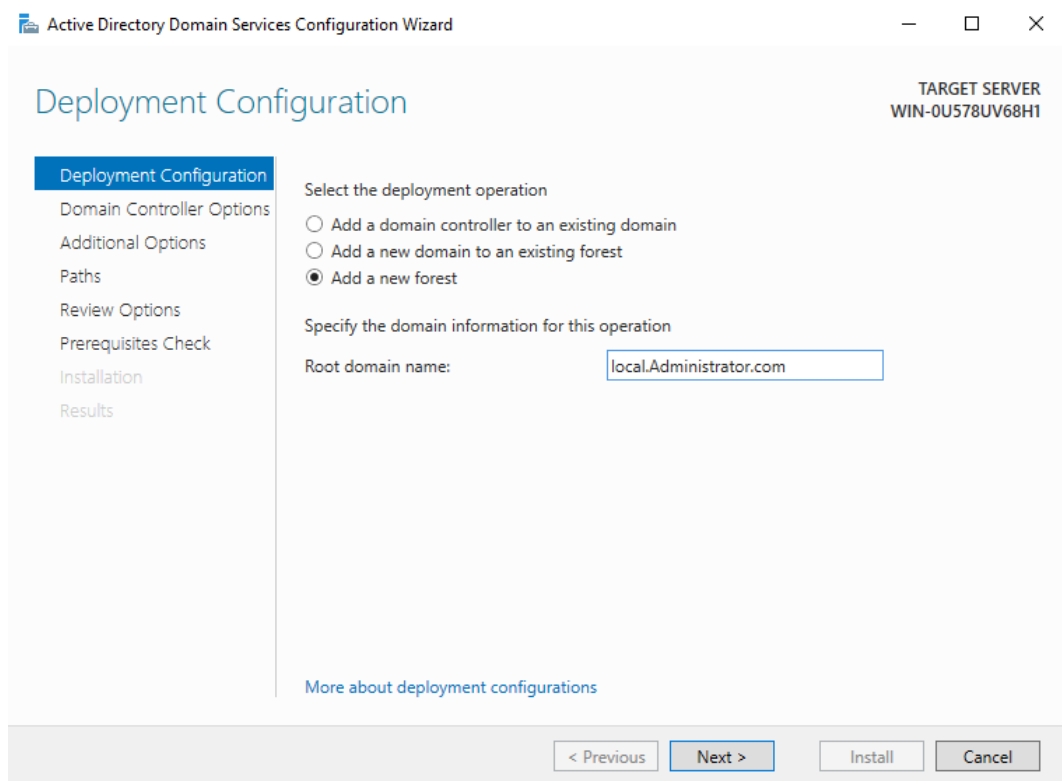
1. Open the **Server Manager** dashboard.



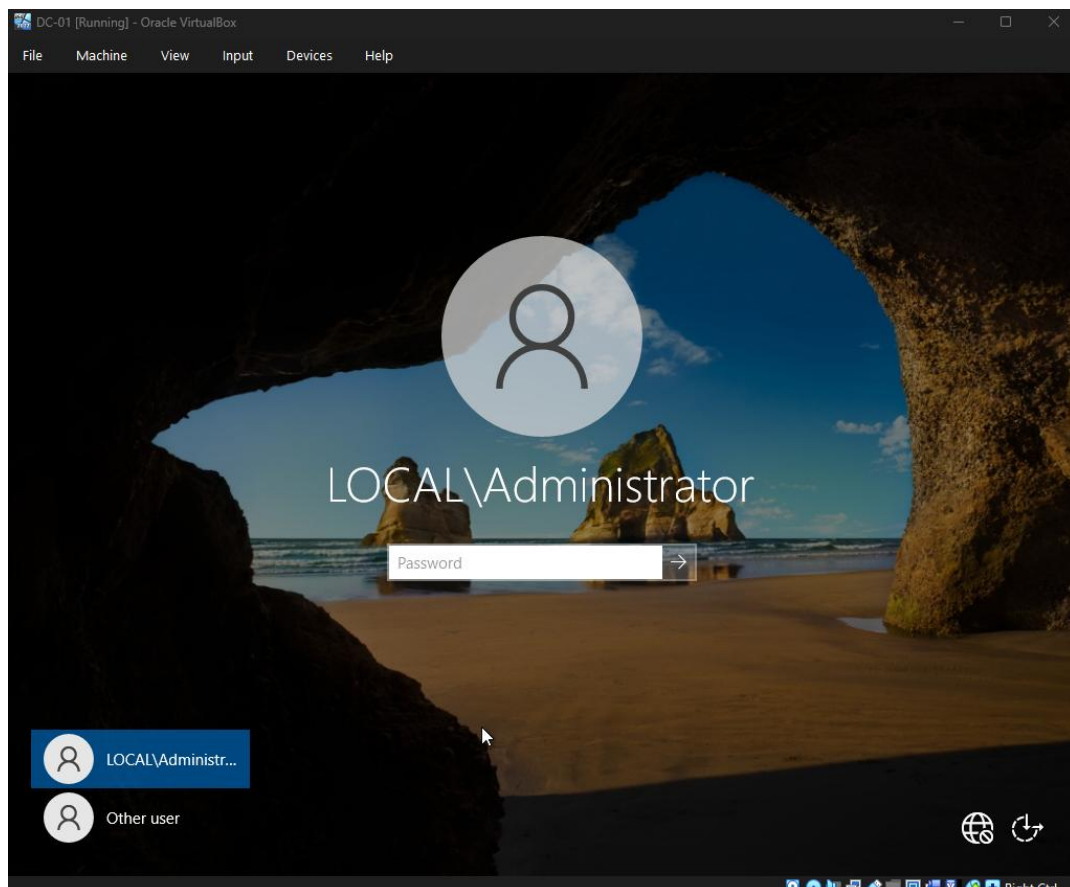
2. Click **Add roles and features**.
3. Click 'Next' until you reach the Server Roles page. Check the box for **Active Directory Domain Services** (AD DS). Accept the additional required features and install.



- Once the installation finishes, you will see a yellow warning flag at the top of Server Manager. Click it and select **Promote this server to a domain controller**.
- Choose **Add a new forest**. Name your root domain (e.g., local.yourname.com).



- Complete the wizard, let the server restart, and you now have a functioning Active Directory.



That LOCAL\Administrator prompt means your promotion to a Domain Controller was successful.

When you promote a server, the local Administrator account is automatically converted into the **Domain Administrator** account.

The Password: Simply enter the exact same Administrator password you created when you first installed Windows Server.

The Prefix: LOCAL\ (or whatever you named your domain) indicates that you are no longer logging into just the local machine; you are logging into the entire directory domain.

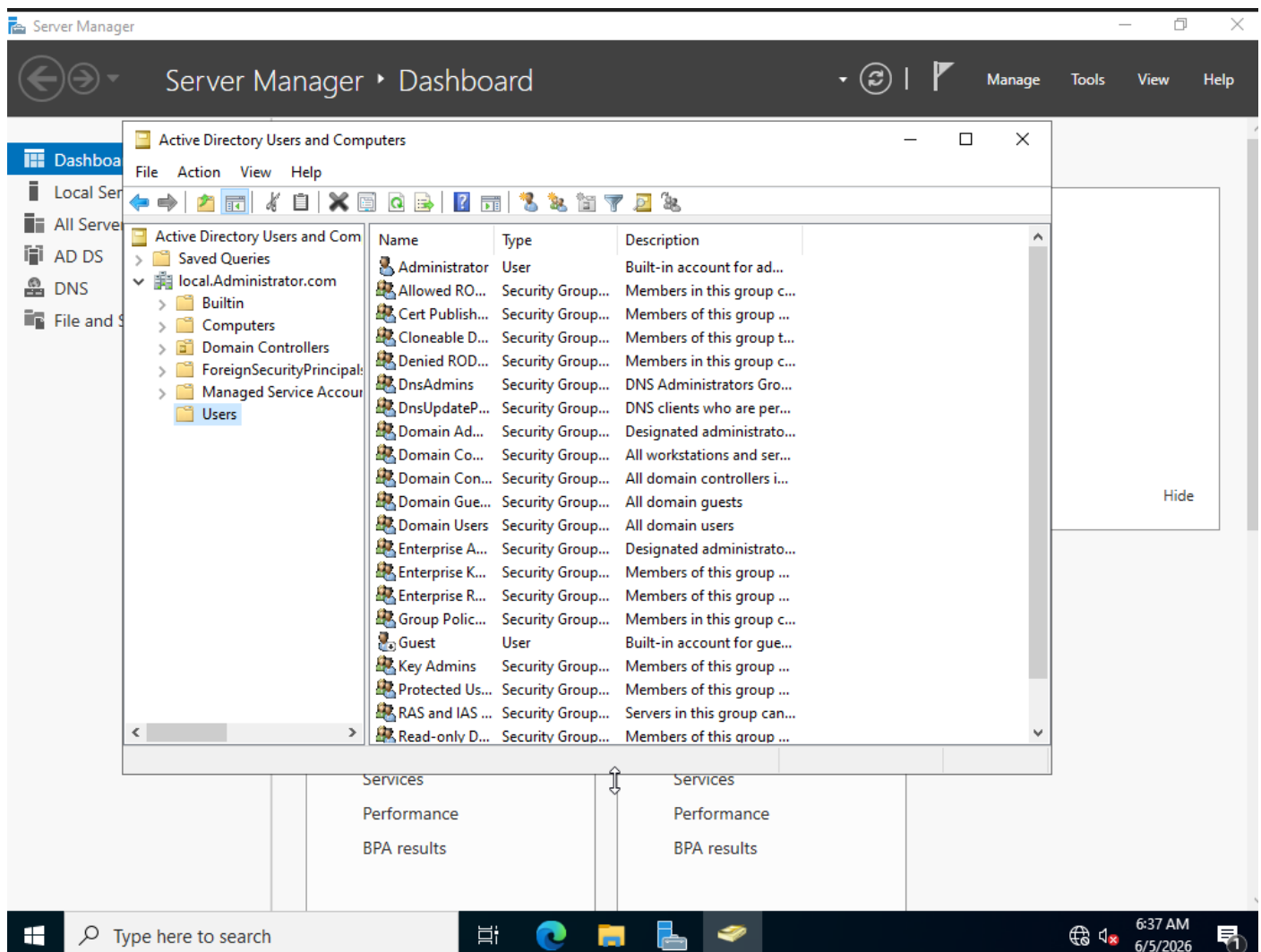
Phase 2: On-Premise Directory Setup and WinRM Bridge

Structuring Your Active Directory

In a real enterprise environment, users aren't just dumped into one folder. They are organized into Organizational Units (OUs) so that Group Policies and access rights can be applied securely.

1: Open Active Directory Users and Computers (ADUC)

1. In Server Manager, click on **Tools** in the top right corner.
2. Select **Active Directory Users and Computers** from the drop-down menu.
3. In the left pane, you will see your domain name (e.g., local.com). Click the little arrow next to it to expand the tree.



2: Create the Main Organizational Unit We are going to create a dedicated sandbox for this project so it stays clean.

1. Right-click on your domain name.
2. Hover over **New** and select **Organizational Unit**.
3. Name it Enterprise_Users and click OK.

3: Create Departmental OUs Now, let's create sub-folders inside Enterprise_Users for different departments. This will be crucial for our automation script later.

1. Right-click the new Enterprise_Users OU.
2. Hover over **New > Organizational Unit**.
3. Create three separate OUs named:
 - IT_Department
 - HR_Department
 - Terminated_Users (This one is specifically for the Automated De-provisioning Phase).

4: Create a Manual Test User To prove we understand the manual provisioning process before we automate it:

1. Right-click on the IT_Department OU.
2. Select **New > User**.
3. Fill in the details (e.g., First Name: Admin, Last Name: Test, User logon name: admin.test).
4. Click Next, assign a temporary password, and check the box that says "**User must change password at next logon**" (a standard IAM security practice).
5. Click Next, then Finish.

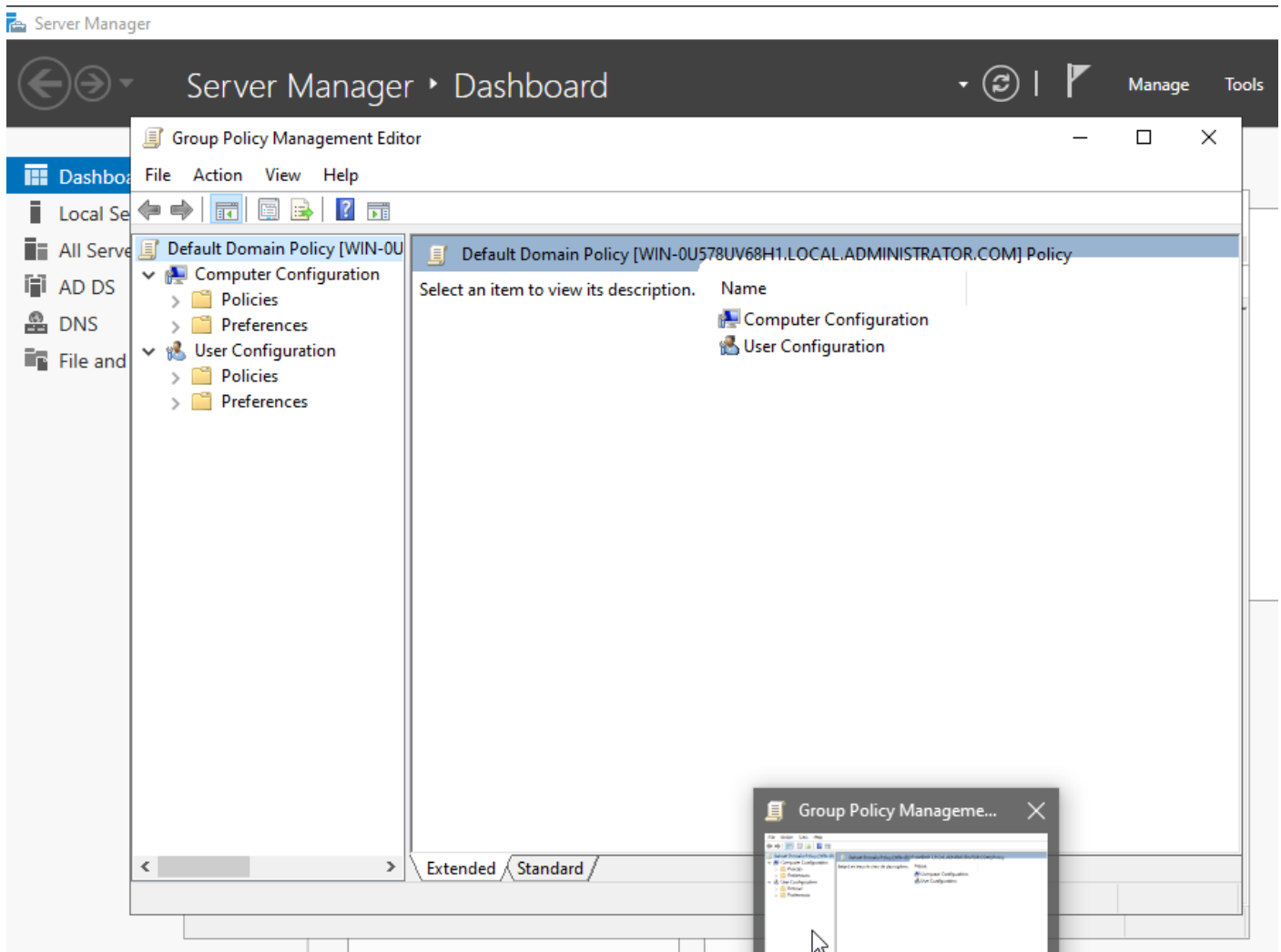
The screenshot shows the 'New Object - User' dialog box in the Active Directory Users and Computers console. The 'Create in' field is set to 'administrator.com/Enterprise_Users/IT_Department'. The 'First name' field contains 'admin', 'Last name' contains 'test', and 'Full name' is 'admin test'. The 'User logon name' is 'admin.test' and the domain dropdown is '@local.Administrator.com'. The 'User logon name (pre-Windows 2000)' is 'LOCAL\admin.test'. The 'Next >' button is highlighted in blue.

We now have a fully functioning, structured on-premise Active Directory environment.

Access Control & Security Auditing

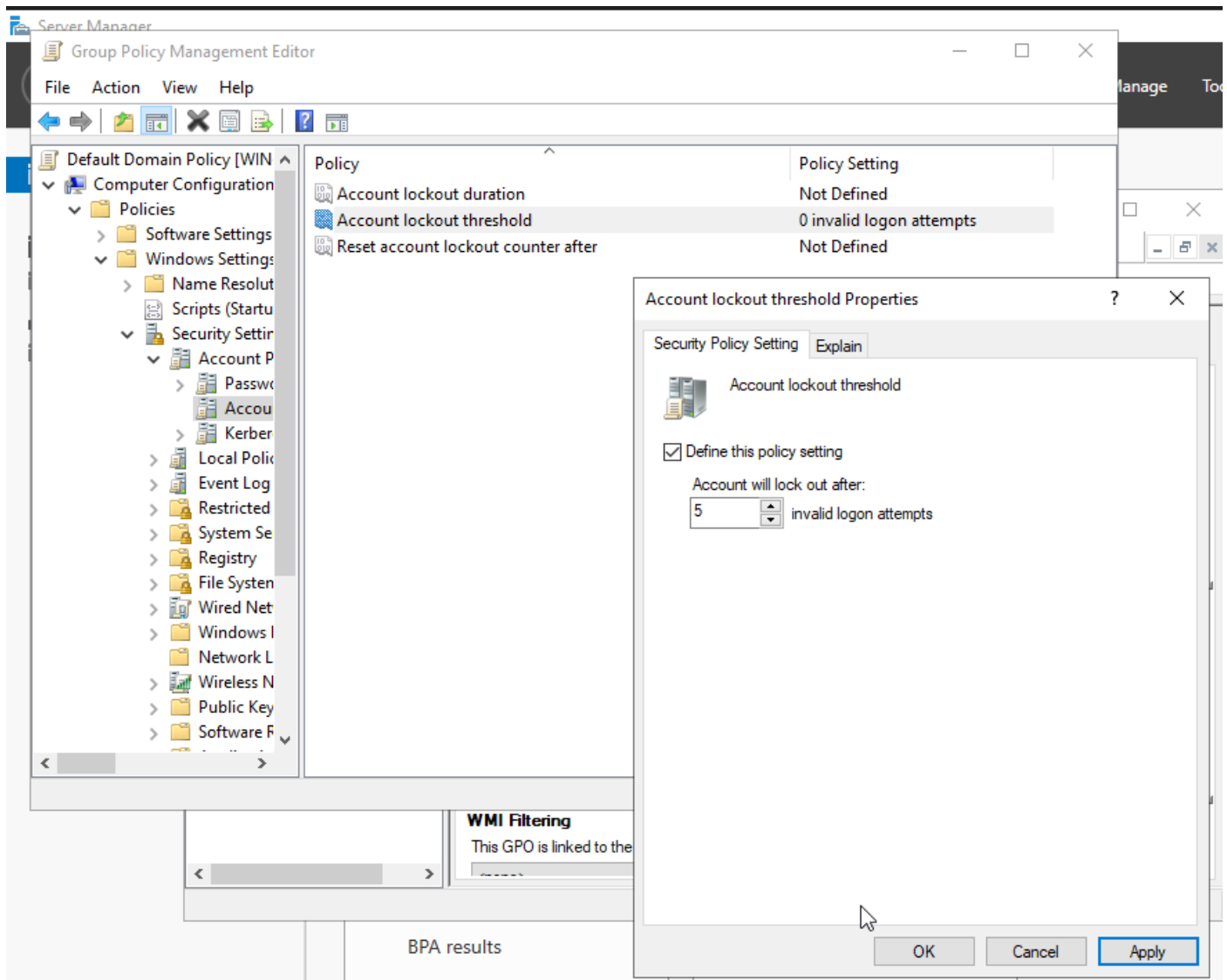
Step 1: Open Group Policy Management

1. Inside your Windows Server VM, open **Server Manager**.
2. Click **Tools** in the top right and select **Group Policy Management**.
3. Expand your forest (Forest: local.Administrator.com) > **Domains** > local.Administrator.com.
4. Right-click on **Default Domain Policy** and select **Edit**. This opens the Group Policy Management Editor.



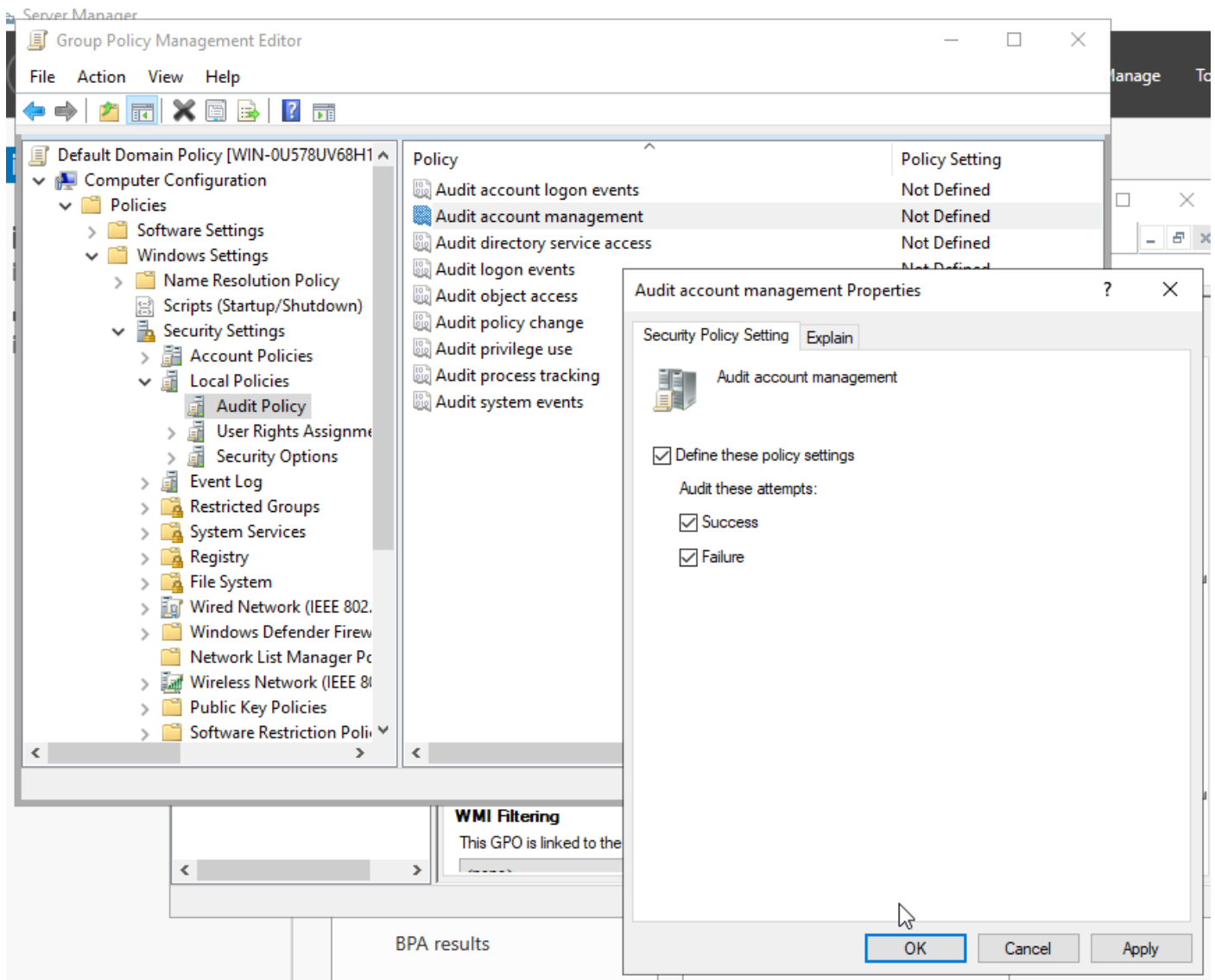
Step 2: Enforce Account Lockout Policies A core IAM responsibility is protecting user accounts from brute-force password attacks.

1. In the editor, navigate to: **Computer Configuration** > **Policies** > **Windows Settings** > **Security Settings** > **Account Policies** > **Account Lockout Policy**.
2. Double-click **Account lockout threshold**.
3. Check "Define this policy setting" and set it to **5 invalid logon attempts**.
4. Click OK. It will automatically suggest setting the lockout duration and observation window to 30 minutes. Accept these defaults.



Step 3: Enable Identity Auditing (SOC Integration Hook) We need to ensure that every time an account is accessed or modified, it is logged.

1. Still in the editor, navigate to: **Computer Configuration > Policies > Windows Settings > Security Settings > Local Policies > Audit Policy**.
2. Double-click **Audit account management** (this tracks when users are created, deleted, or put into groups). Check both **Success** and **Failure**.
3. Double-click **Audit logon events** (this tracks successful logins and failed login attempts). Check both **Success** and **Failure**.
4. Close the Group Policy Management Editor.



Step 4: Force the Policy Update To make these rules apply immediately across your entire domain sandbox:

1. Open Command Prompt or PowerShell inside the VM.
2. Run the command: `gpupdate /force`

You have now successfully engineered a hardened identity perimeter with full audit tracking.

```
PS C:\Users\Administrator> gpupdate /force
Updating policy...

Computer Policy update has completed successfully.
User Policy update has completed successfully.

PS C:\Users\Administrator>
```

Set Up the Automation Bridge (WinRM)

To allow a Python script running on your physical machine to control Active Directory inside the VirtualBox VM, we need to enable **Windows Remote Management (WinRM)** on your Windows Server.

1. Inside your Windows Server VM, open **PowerShell** as an Administrator.
2. Run the following command to configure WinRM to accept remote management traffic:

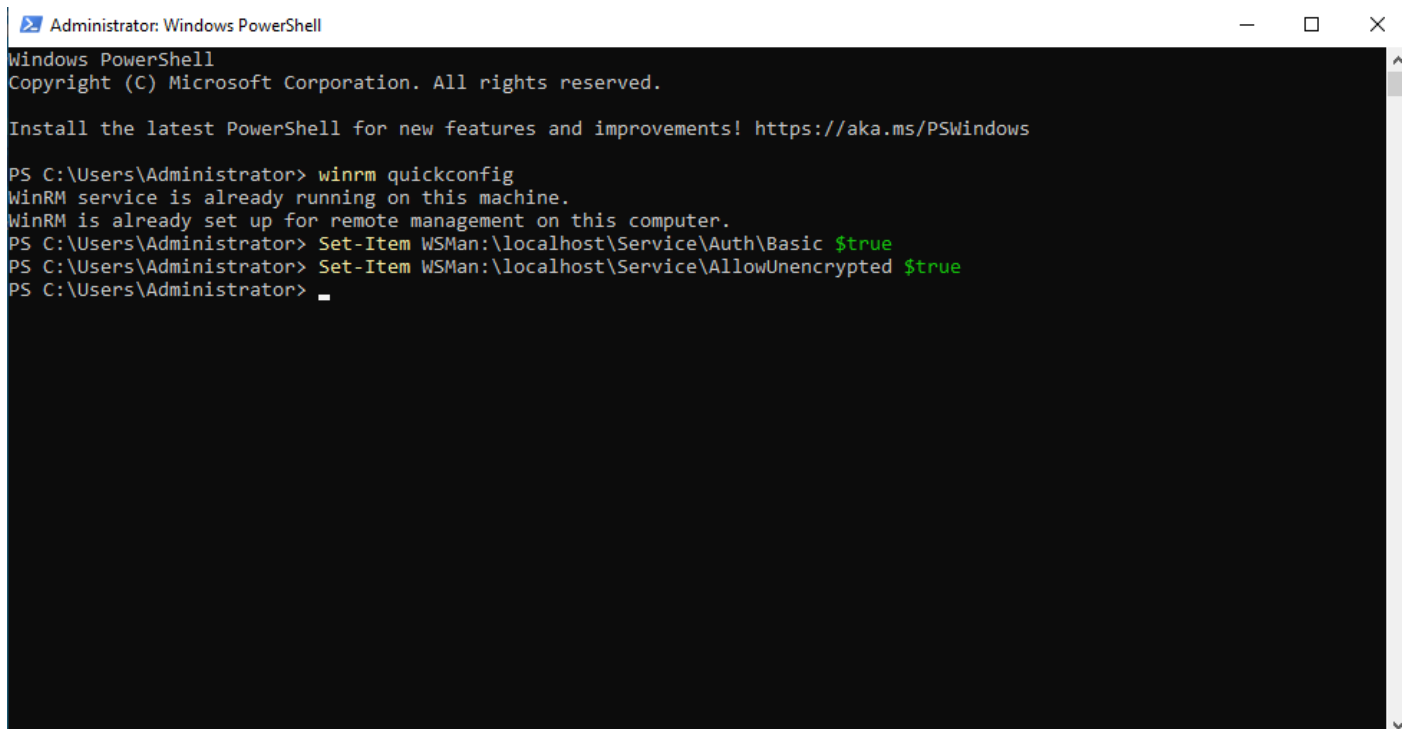
```
winrm quickconfig
```

(Press Y to accept any prompts).

3. Next, let's ensure it allows unencrypted traffic over your local private network interface for testing purposes, and allows basic authentication:

```
Set-Item WSMan:\localhost\Service\Auth\Basic $true
```

```
Set-Item WSMan:\localhost\Service\AllowUnencrypted $true
```



```
Administrator: Windows PowerShell
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\Administrator> winrm quickconfig
WinRM service is already running on this machine.
WinRM is already set up for remote management on this computer.
PS C:\Users\Administrator> Set-Item WSMan:\localhost\Service\Auth\Basic $true
PS C:\Users\Administrator> Set-Item WSMan:\localhost\Service\AllowUnencrypted $true
PS C:\Users\Administrator>
```

Configure VM Network Settings

By default, VirtualBox puts your VM behind a NAT network, which isolates it from your host machine. We need to let your host talk to it.

1. Shut down your Windows Server VM safely.
2. In the VirtualBox Manager, select your DC01 VM and open its **Settings**.
3. Go to **Network > Adapter 1**.
4. Change "Attached to" from **NAT** to **Bridged Adapter** (this will allow the vm to share your home router's network).
5. Start the VM back up, log in, open a command prompt, and run ipconfig. Note down the IPv4 address of the server (e.g., 192.168.56.101).

Prepare the Python Environment (Host Machine)

On your main physical machine, we will write the Python script. We will use the **pywinrm** library, which allows Python to run PowerShell commands directly on your remote Windows Server.

1. Open your terminal or command prompt on your host computer.
2. Install the necessary library:

```
pip install pywinrm
```

Crafting the Provisioning Pipeline

Here is the exact framework for our automation script. It mimics a real HR onboarding pipeline. We define a data structure of new hires (acting as our database), and then use a Python loop to generate the specific PowerShell commands needed to build the accounts inside their correct OUs.

Create a file named `provision_users.py` on your host machine and paste the following code:

```
import winrm
# 1. Connection Details to your Active Directory VM
# VM's IP address and Administrator password (Example Credentials)

VM_IP = "192.168.x.x"
USERNAME = r"Administrator@local.Administrator.com"
PASSWORD = r"Password@123"

# 2. Mock HR New Hire Database
new_hires = [
    {"first": "Alice", "last": "Smith", "dept": "IT_Department", "title": "Cloud Engineer"},
    {"first": "Bob", "last": "Jones", "dept": "HR_Department", "title": "HR Specialist"},
    {"first": "Charlie", "last": "Miller", "dept": "IT_Department", "title": "Security Analyst"}
]

def provision_user(session, user):
    first_name = user["first"]
    last_name = user["last"]
    dept = user["dept"]
    title = user["title"]

    # Construct standard enterprise usernames: e.g., jsmith
    sam_account_name = f"{first_name[0].lower()}{last_name.lower()}"
    user_principal_name = f"{sam_account_name}@local.Administrator.com"
    display_name = f"{first_name} {last_name}"

    print(f"Creating account for {display_name} in {dept}...")

    # Raw PowerShell script to execute inside the Windows Server VM
    # This creates the user, sets a default secure password, enables the account, and puts it
    in the correct OU
    ps_script = f"""
    $passwd = ConvertTo-SecureString "Welcome2026!" -AsPlainText -Force
    New-ADUser -Name "{display_name}" `
        -GivenName "{first_name}" `
        -Surname "{last_name}" `
        -SamAccountName "{sam_account_name}" `
```

```

        -UserPrincipalName "{user_principal_name}" `
        -Path "OU={dept},OU=Enterprise_Users,DC=local,DC=Administrator,DC=com" `
        -Title "{title}" `
        -AccountPassword $passwd `
        -Enabled $true `
        -ChangePasswordAtLogon $true
    """

# Run the command over the WinRM bridge
result = session.run_ps(ps_script)

if result.status_code == 0:
    print(f"Successfully provisioned: {sam_account_name}")
else:
    print(f"Error provisioning {sam_account_name}: {result.std_err.decode('utf-8')}")

def main():
    # Establish connection session
    session = winrm.Session(f"http://{VM_IP}:5985/wsman", auth=(USERNAME, PASSWORD),
transport='ntlm')

    # Process the queue
    for user in new_hires:
        provision_user(session, user)

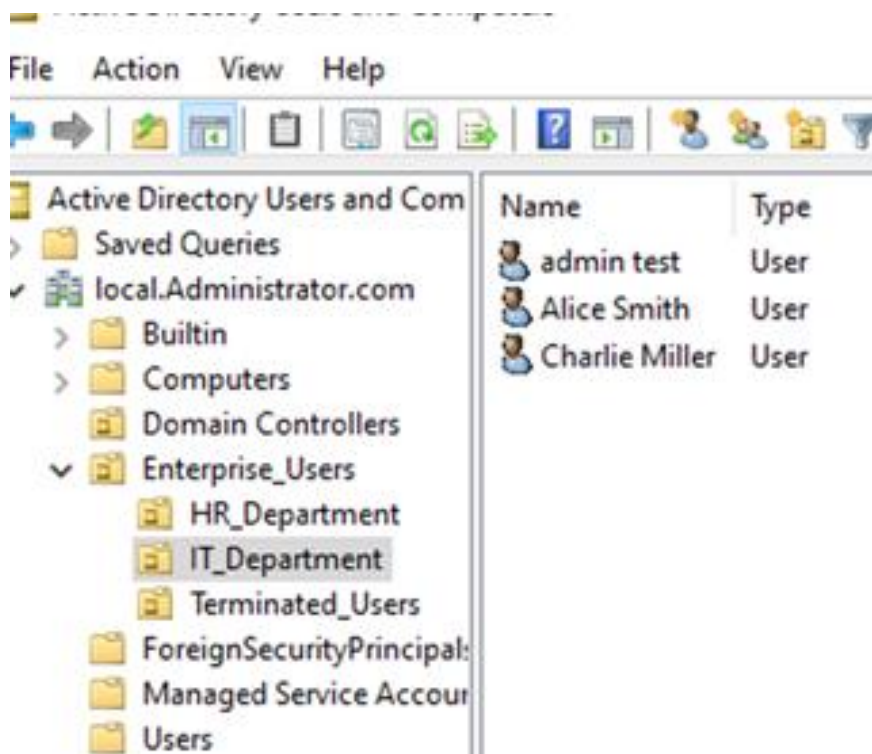
if __name__ == "__main__":
    main()

```

Run the script on your host machine: `python provision_users.py`.

Once it finishes successfully, go back into your Windows Server VM, open **Active Directory Users and Computers**, and hit **Refresh**.

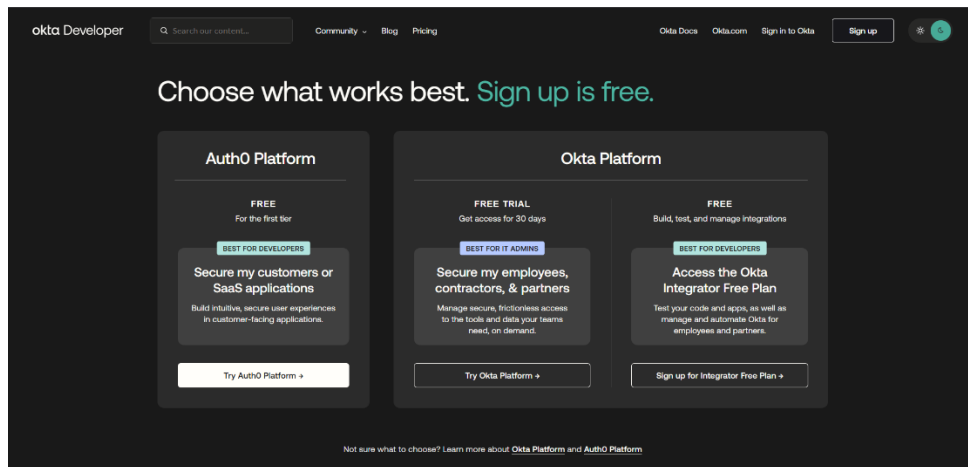
You will see Alice, Bob, and Charlie sitting neatly inside their respective department folders.



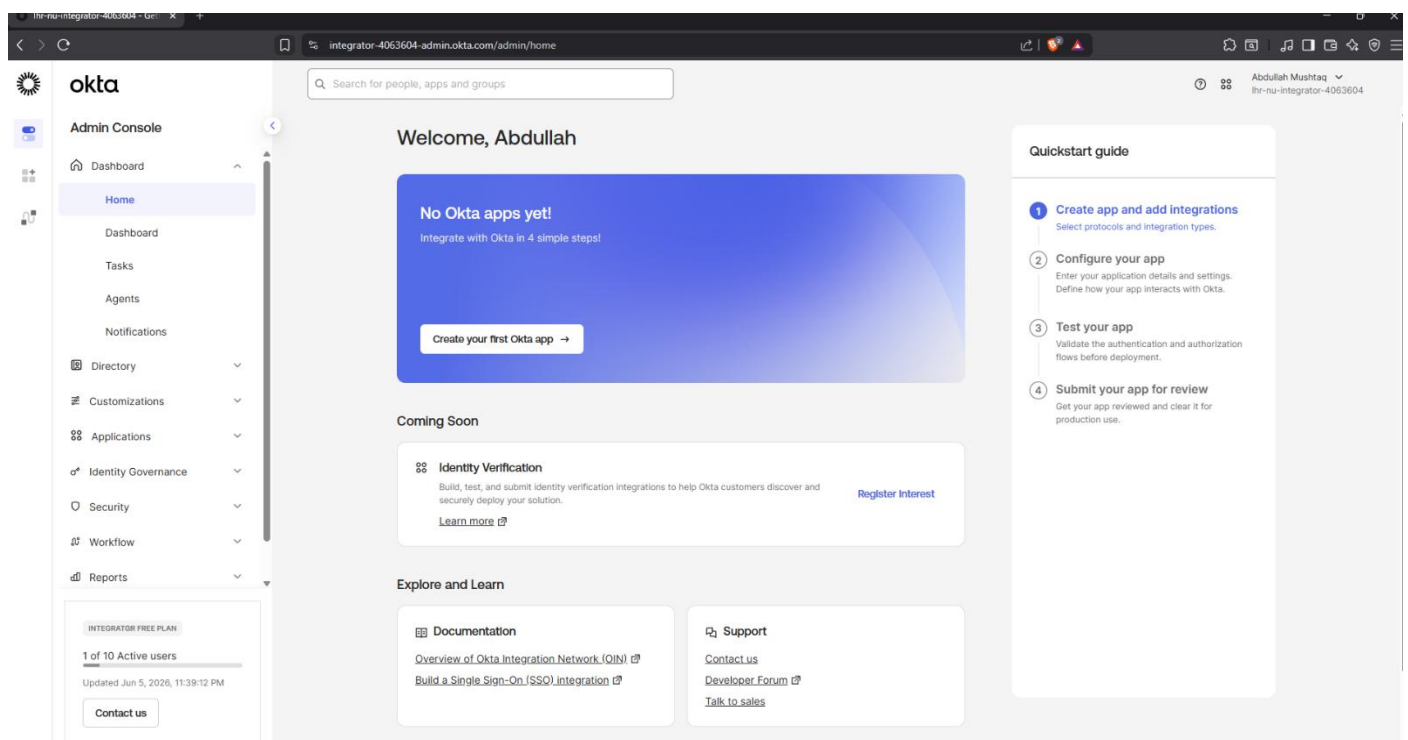
Phase 3: Cryptographic Alignment and Agent Deployment

Create Your Free Okta Tenant

1. On your physical host machine, go to developer.okta.com/signup.
2. Select the “Sign up for integration free plan” and use your work email or a student mail.



3. Once registered, check your email. Okta will send you a welcome email containing your **Okta Domain URL** (it will look something like dev-12345678.okta.com) and your sign-in details.
4. Log into your new Okta dashboard. Ensure you click the **Admin** button in the top right corner to access the Okta Admin Console.

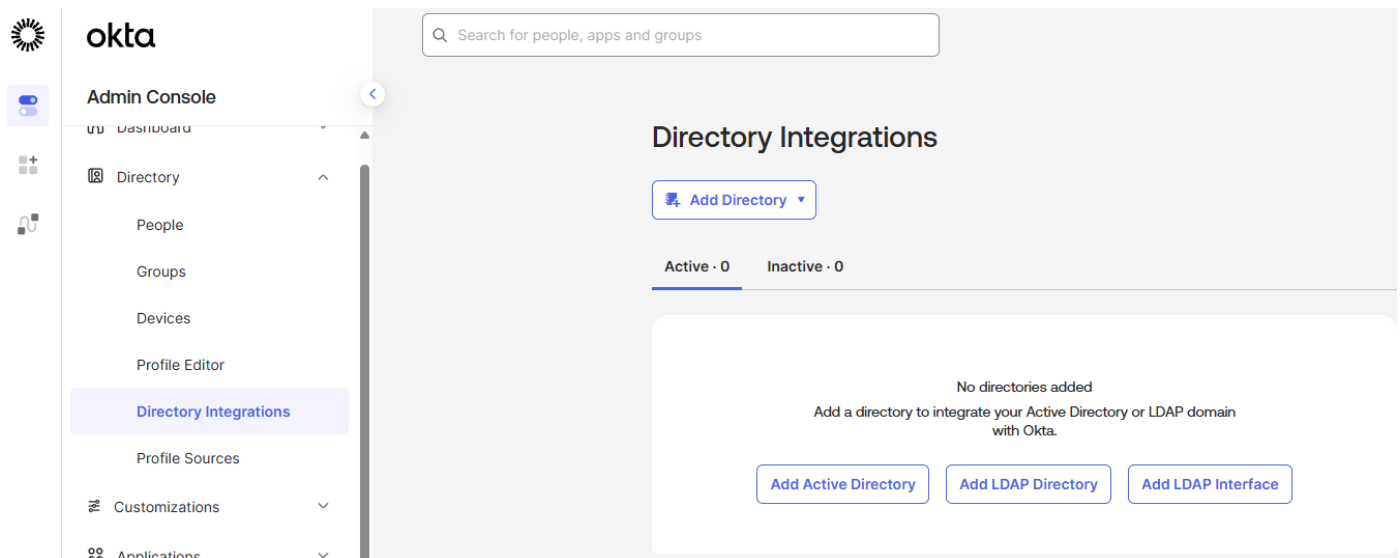


Prepare the Integration

We need to tell Okta to expect an incoming connection from your Windows Server.

1. In the Okta Admin Console, look at the left-hand menu. Navigate to **Directory > Directory Integrations**.
2. Click **Add Directory > Add Active Directory**.

3. Review the quick setup requirements and click **Add Active Directory**.



4. You will be given a button to **Download Agent**. Click it to download the Okta AD Agent installer.

Force TLS 1.2 Encryption

We need to force Windows Server's .NET framework to use modern TLS 1.2 encryption so Okta's cloud accepts the connection.

1. Open **PowerShell as Administrator** inside your Windows Server VM.
2. Copy and paste the following three commands, hitting Enter after each one:

```
Set-ItemProperty -Path
```

```
'HKLM:\SOFTWARE\Wow6432Node\Microsoft\.NETFramework\v4.0.30319' -Name  
'SchUseStrongCrypto' -Value '1' -Type DWord
```

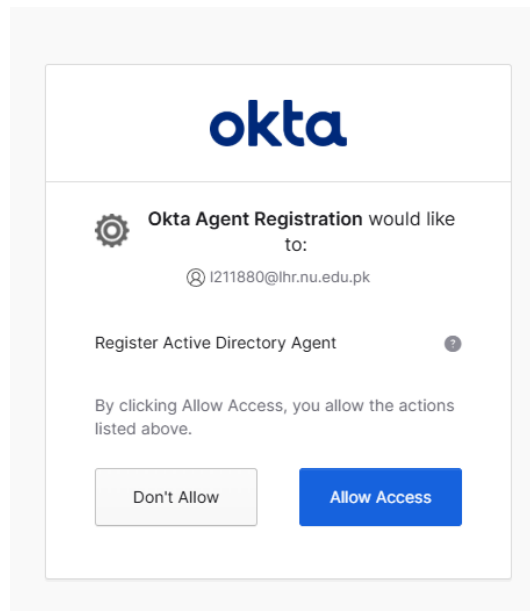
```
Set-ItemProperty -Path 'HKLM:\SOFTWARE\Microsoft\.NETFramework\v4.0.30319' -  
Name 'SchUseStrongCrypto' -Value '1' -Type DWord
```

```
[Net.ServicePointManager]::SecurityProtocol = [Net.SecurityProtocolType]::Tls12
```

(If it asks for confirmation on any of them, type Y and press Enter).

Okta AD Agent Setup

1. Transfer the Okta AD Agent installer to your DC01 Windows Server VM (you can drag and drop if VirtualBox Guest Additions is installed, or just open a browser inside the VM, log into your Okta admin console, and download it directly there).
2. Run the installer inside the VM.
3. When prompted for your Okta Org URL, enter the dev-xxxxxxx.okta.com address from your welcome email.
4. The installer will open a browser window asking you to authenticate. Log in with your Okta Admin credentials and click **Allow Access** to grant the agent permissions.

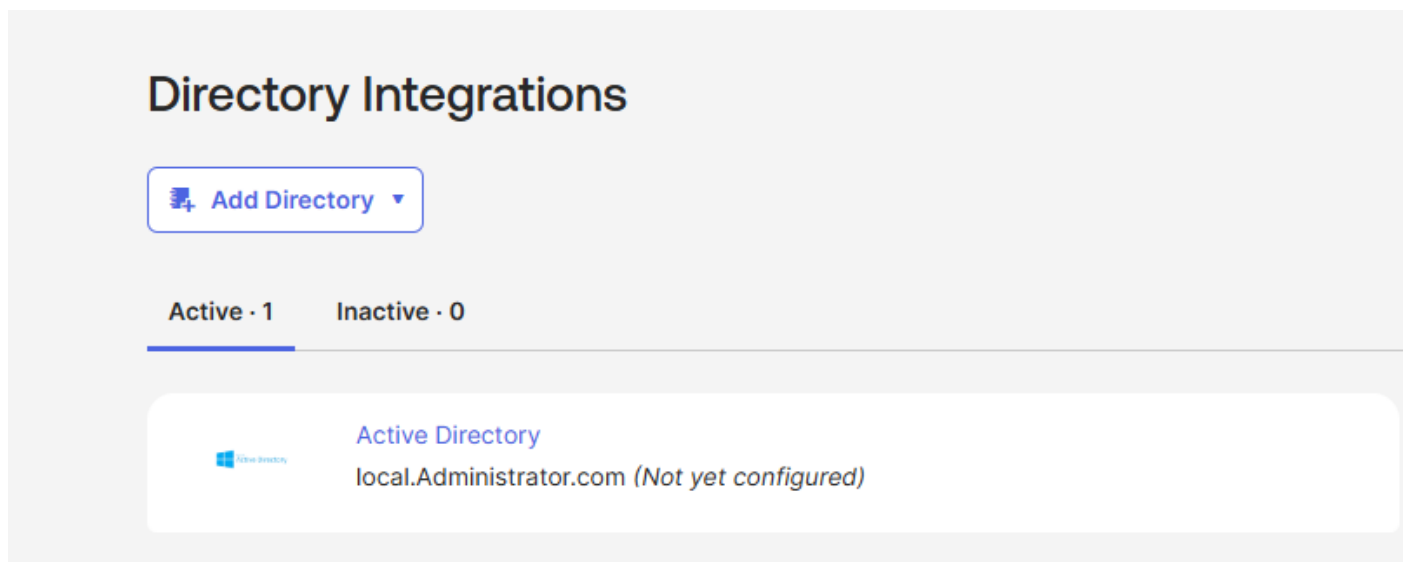


5. Finish the installation. The agent is now actively talking to the Okta cloud.

Phase 4: Cloud Identity Federation and Synchronization

Now that the agent has successfully established that secure outbound tunnel from Windows Server VM to the cloud, we can finish the federation process.

Switch back over to physical host machine's web browser, where you have the Okta Admin Console open (<https://integrator-4063604-admin.okta.com>). The setup page should either have refreshed, or you can find it by navigating to **Directory** > **Directory Integrations** > **Active Directory**.



Here is how we configure the sync rules to safely import Alice, Bob, and Charlie into your cloud directory.

Select the Organizational Units (OUs) to Sync

Okta will look at your entire Active Directory tree. We want to be highly selective so we don't clutter your cloud environment with built-in system accounts.

1. In the Okta setup wizard (or under the **Provisioning** tab > **To Okta** settings), look for the **Nodes** or **OUs** section.
2. You will see a checkbox tree of your domain (local.Administrator.com).
3. Uncheck the root box so nothing is selected by default.
4. Scroll down and check **only** the box next to **Enterprise_Users** (the OU containing Alice, Bob, and Charlie).
5. Click **Next** (Make sure we are using UPN option for username format).

2 Connect an Organizational Unit to Okta

Select the Organizational Units (OUs) that you'd like to sync Users from:

☐ dc=local,dc=administrator,dc=com

- ☐ computers
- ☐ foreignsecurityprincipals
- ☐ managed service accounts
- ☐ users
- ☐ domain controllers
- ☒ enterprise_users
 - ☒ hr_department
 - ☒ it_department
 - ☒ terminated_users

Show more

Select the Organizational Units (OUs) that you'd like to sync Groups from:

☐ dc=local,dc=administrator,dc=com

- ☐ computers
- ☐ foreignsecurityprincipals
- ☐ managed service accounts
- ☐ users
- ☐ domain controllers
- ☒ enterprise_users
 - ☒ hr_department
 - ☒ it_department
 - ☒ terminated_users

Show more

Okta username format

User Principal Name (UPN)

Select the username that people imported from AD use to log into Okta.

Next »

Handle Attribute Mappings

1. Then the **Attribute Mapping** stage is displayed. This is where you tell Okta which active data fields (like phone numbers, job titles, or department names) you want to pull from your Windows Server into user profiles in the cloud. For an enterprise deployment, engineers use this to sync custom fields. But for our lab sandbox, the default fields Okta has already pre-

selected for you on the right side (under *Base Schema* and *Custom Schema*) are more than enough to handle Alice, Bob, and Charlie.

3 Select the attributes to build your Okta User profile

Refresh Attribute List

Next »

☐	Attribute Name	Type	Description
☐	USNIntersite	integer	USN-Intersite
☐	aCSPolicyName	string	ACS-Policy-Name
☐	accountExpires	string	Account-Expires
☐	accountNameHistory	array	Account-Name-History
☒	countryCode	integer	Country-Code
☐	adminCount	integer	Admin-Count
☐	adminDescription	string	Admin-Description
☐	adminDisplayName	string	Admin-Display-Name
☐	altSecurityIdentities	array	Alt-Security-Identities
☐	assistant	string	Assistant
☐	attributeCertificateAttribute	array	A digitally signed or certified identity and set of attributes. Used to bind authorization information to an identity. X.509
☐	audio	array	The Audio attribute type allows the storing of sounds in the Directory.
☐	badPasswordTime	string	Bad-Password-Time
☐	badPwdCount	integer	Bad-Pwd-Count
☐	bridgeheadServerListBL	array	Bridgehead-Server-List-BL
☐	businessCategory	array	Business-Category
☐	canonicalName	array	Canonical-Name
☐	carLicense	array	Vehicle license or registration plate.
☒	l	string	Locality-Name
☒	cn	string	Common-Name
☒	co	string	Text-Country
☐	codePage	integer	Code-Page
☐	comment	string	User-Comment
☐	company	string	Company
☐	controlAccessRights	array	Control-Access-Rights
☒	c	string	Country-Name

Imported Attributes

Base Schema (required)

sAMAccountName

objectGUID

distinguishedName

userPrincipalName

Custom Schema

countryCode

l

cn

co

c

physicalDeliveryOfficeName

department

departmentNumber

displayName

division

employeeID

employeeNumber

facsimileTelephoneNumber

manager

middleName

mobile

postalCode

preferredLanguage

st

streetAddress

telephoneNumber

description

title

- Simply click the blue **Next »** button in the top right corner of that panel to move forward.
- If you look at the left-hand menu under **Settings**, you will see you are currently on **To App**. In Okta terminology, "To App" means pushing data *from* Okta *down into* Active Directory. Because we aren't doing that, you can completely ignore that yellow warning box.
- On that left-hand menu under Settings, click **To Okta**.

Agents
Provisioning
Import
Push Groups
Assignments

Settings
To App
To Okta
Integration
Password migration

Active Directory

→

okta

General

Edit

Import users from Active Directory to create new Okta users. If the Okta user already exists, the two accounts will automatically be linked. Imported users are assigned Active Directory access when they are confirmed on the Import tab.

Schedule import	Never Select never if you prefer to import manually
Okta username format	Custom Select the username users should enter to log into Okta.
Update application username on	Create and update
JIT provisioning	☐ Create and update users on login JIT provisioning requires delegated authentication to be enabled.
USG support	☐ Universal security group support Allows Universal security group membership for user to be imported into Okta. This setting requires JIT Provisioning.
Do not import users	☐ Skip users during import
Activation emails	☐ Don't send new user activation emails for this domain Activation emails can only be suppressed when a domain is using delegated authentication .

- the **Okta username format** is currently set to Custom. We need to change that so they get proper email-style logins.
- Change the **Okta username format** from Custom to **User Principal Name (UPN)** and **Save**.

Select the Target Folder (The missing setting)

Okta actually hides the Organizational Unit (OU) selection under a different tab on the left.

- Look at the left-hand menu under **Settings** and click on **Integration** (right below where you currently have "To Okta" selected).
- Scroll down until you see the **Organizational Units (OUs) that will sync and import** section.
- Uncheck the main local.Administrator.com box to clear the default selection.
- Expand the tree and check **only** the box next to **Enterprise_Users**.

5. Click **Save**.

Run the Import

Now that Okta knows *who* to look for and *how* to format their names, we pull the trigger.

1. Click the **Import** tab on the main top menu (next to Provisioning).

Import Now Clear Unconfirmed Users 0 imported users need review · 0 imported users confirmed

ALL NO EXACT PARTIAL IGNORED Search Confirm Assignments 0

Show 10 First Previous Next Last

Imported User okta Okta User Assignment ☒

No matching records found

Show 10 First Previous Next Last

Confirm Assignments 0

2. Click **Import Now**.
3. Select **Full Import** and click **Standard Import**.

Importing users from Active Directory...

0%

Importing Application Data...

4 users scanned!

- 4 new users imported
- 0 existing users updated
- 0 existing users unchanged
- 0 users removed

0 groups scanned!

OK

Once the scan finishes, it should show 4 users found. Check the boxes next to their names and hit **Confirm Assignments** to officially bring them into the cloud.

Import Now

Clear Unconfirmed Users

4 imported users need review · 0 imported users confirmed

ALL

NO

EXACT

PARTIAL

IGNORED

Search

Confirm Assignments 4

Show 10 Showing 1 - 4 of 4

First

Previous

1

Next

Last

Active Directory	Imported User	Okta	Okta User Assignment	
	NO Okta user matches found Name: Bob Jones Username: bjones@local.Administrator.com Email:	ASSIGN TO →	NEW Okta user Name: Bob Jones Username: bjones@local.Administrator.com Email: bjones@local.Administrator.com	☒
	NO Okta user matches found Name: Charlie Miller Username: cmiller@local.Administrator.com Email:	ASSIGN TO →	NEW Okta user Name: Charlie Miller Username: cmiller@local.Administrator.com Email: cmiller@local.Administrator.com	☒
	NO Okta user matches found Name: Alice Smith Username: asmith@local.Administrator.com Email:	ASSIGN TO →	NEW Okta user Name: Alice Smith Username: asmith@local.Administrator.com Email: asmith@local.Administrator.com	☒
	NO Okta user matches found Name: admin test Username: admin.test@local.Administrator.com Email:	ASSIGN TO →	NEW Okta user Name: admin test Username: admin.test@local.Administrator.c... Email: admin.test@local.Administrator.c...	☒

Show 10 Showing 1 - 4 of 4

First

Previous

1

Next

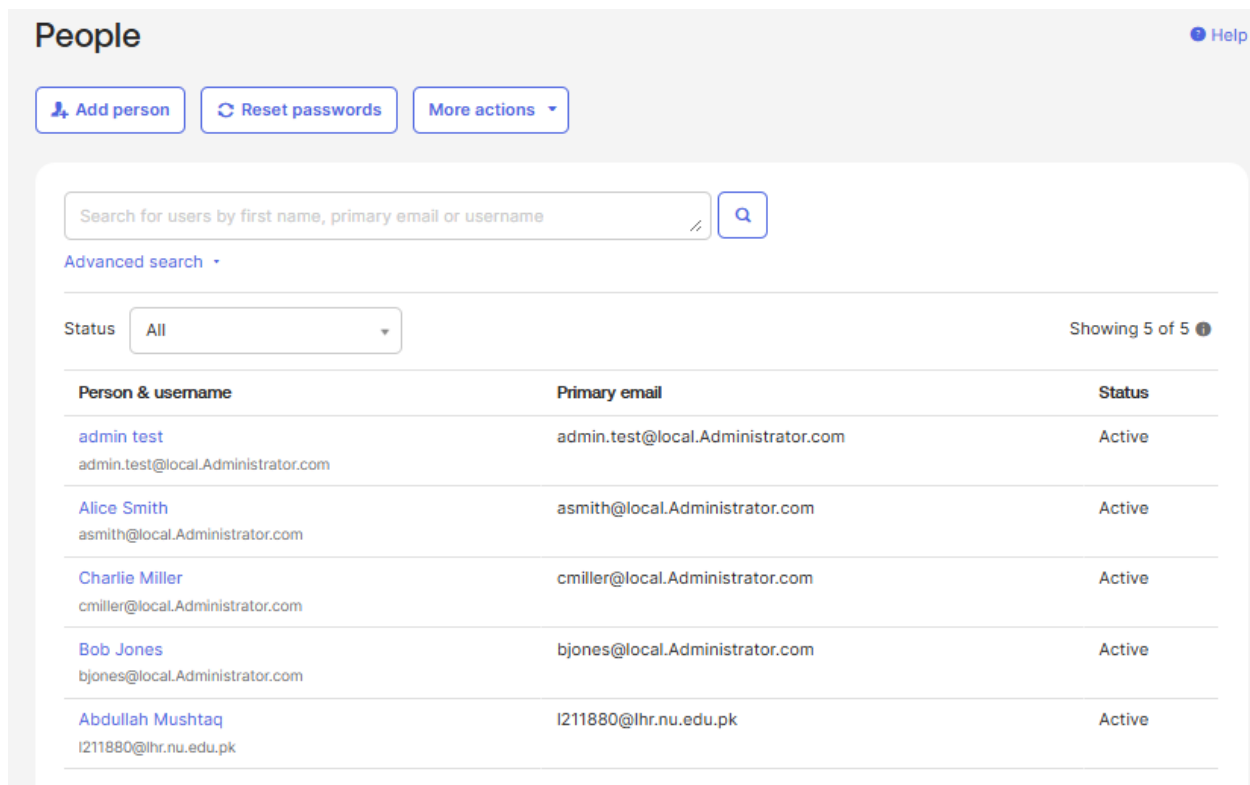
Last

Confirm Assignments 4

Finalize the Cloud Activation

1. Click that blue **Confirm Assignments 4** button in the bottom right corner.
2. If a popup appears asking if you want to automatically activate them, go ahead and confirm, otherwise you may activate them manually.
3. On the left-hand menu, navigate to **Directory > People**.
4. You will now see Alice, Bob, Charlie, and your test admin listed as fully active Okta users.

Because we have Okta connected to Active Directory, Okta is now treating our Windows Server as the "Source of Truth." If you change a password, update a job title, or disable an account in your local Windows Server, Okta will automatically sync that change to the cloud.



Phase 5: De-provisioning Automation and Incident Response Readiness

Creating users is only half the job. When an employee leaves a company, leaving their account active is a massive security vulnerability.

In a modern enterprise environment, you rarely disable accounts manually by clicking through Windows Server menus. Ideally, you have your SIEM catch a critical security alert (like a compromised credential), fire a webhook to a SOAR platform, and have that pipeline automatically trigger a Python script to isolate and disable the AD account instantly.

We are going to write that Python script and use it to target and "offboard" the admin test account.

The Python Offboarding Script

On your physical host machine, create a new file named `offboard_user.py` and drop in this code. You will need to update the IP address and password to match your lab environment, just like you did in Phase 2.

```
import winrm

# --- Enterprise Environment Configuration ---
target_ip = "192.168.18.149" # Replace with your DC01 IP address
admin_user = "Administrator@local.Administrator.com"
admin_pass = "Password@123" # Replace with your actual Domain Admin password

# The target user we want to disable (using the UPN)
user_to_disable = "admin.test@local.Administrator.com"

# --- PowerShell Payload ---
# This script disables the account and hides it from the global address list
```



```

# This explicitly filters by UPN first to bypass Identity identifier limitations
ps_script = f"""
Import-Module ActiveDirectory
$UPN = "{user_to_disable}"
try {{
    # 1. Locate the user by their full UPN
    $ADUser = Get-ADUser -Filter "UserPrincipalName -eq '$UPN'" -ErrorAction Stop

    if ($ADUser) {{
        # 2. Disable the Active Directory account immediately
        $ADUser | Disable-ADAccount -ErrorAction Stop

        # 3. Add a description noting the automated offboarding
        $date = Get-Date -Format "yyyy-MM-dd"
        $ADUser | Set-ADUser -Description "Automatically disabled by Python IAM script
on $date" -ErrorAction Stop

        Write-Output "SUCCESS: Account $UPN has been disabled and locked down."
    }} else {{
        Write-Error "FAILED: No user found matching UPN $UPN."
    }}
}}
catch {{
    Write-Error "FAILED: An error occurred while modifying the account. Details: $_"
}}
"""

print(f"[*] Initiating automated offboarding protocol for: {user_to_disable}")

# Open the WinRM bridge to the server
try:
    session = winrm.Session(target_ip, auth=(admin_user, admin_pass),
transport='ntlm')
    result = session.run_ps(ps_script)

    if result.status_code == 0:
        print("[+] Active Directory Response:")
        print(result.std_out.decode('utf-8').strip())
    else:
        print("[-] Error executing payload:")
        print(result.std_err.decode('utf-8').strip())

except Exception as e:
    print(f"[-] Connection failed: {e}")

```

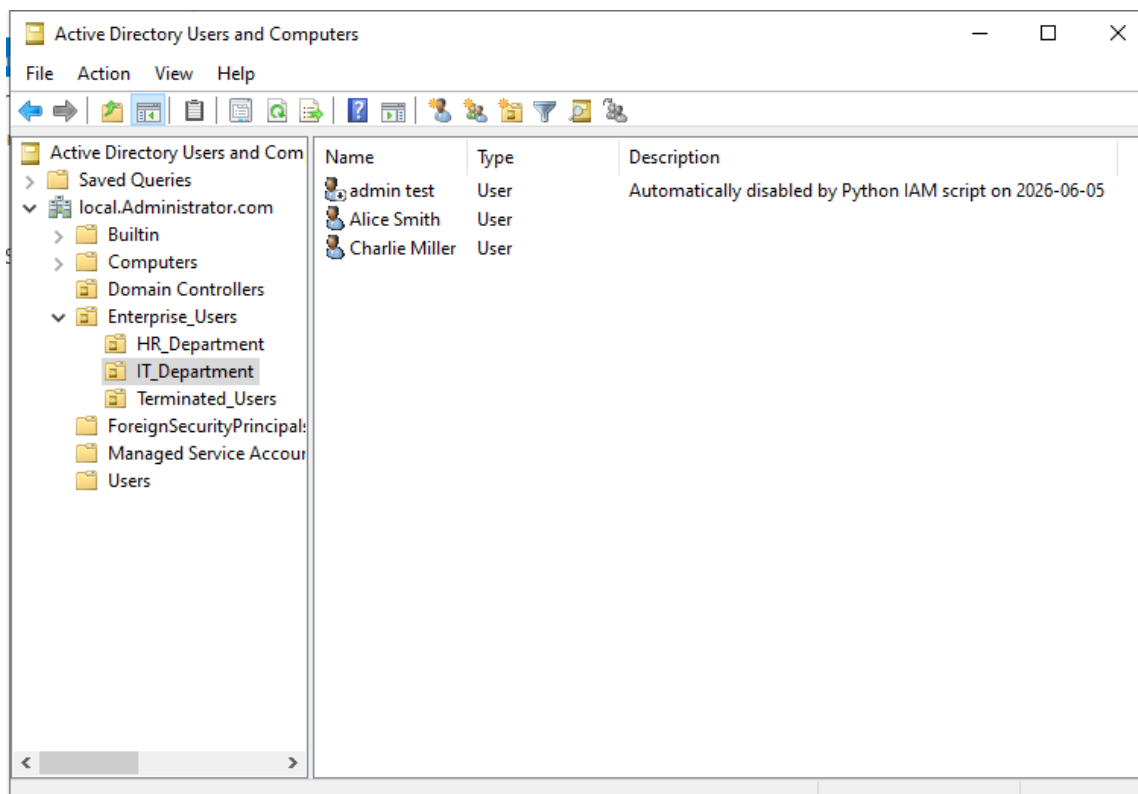
Run the script from your host machine. If it succeeds, jump into your Windows Server VM.

```

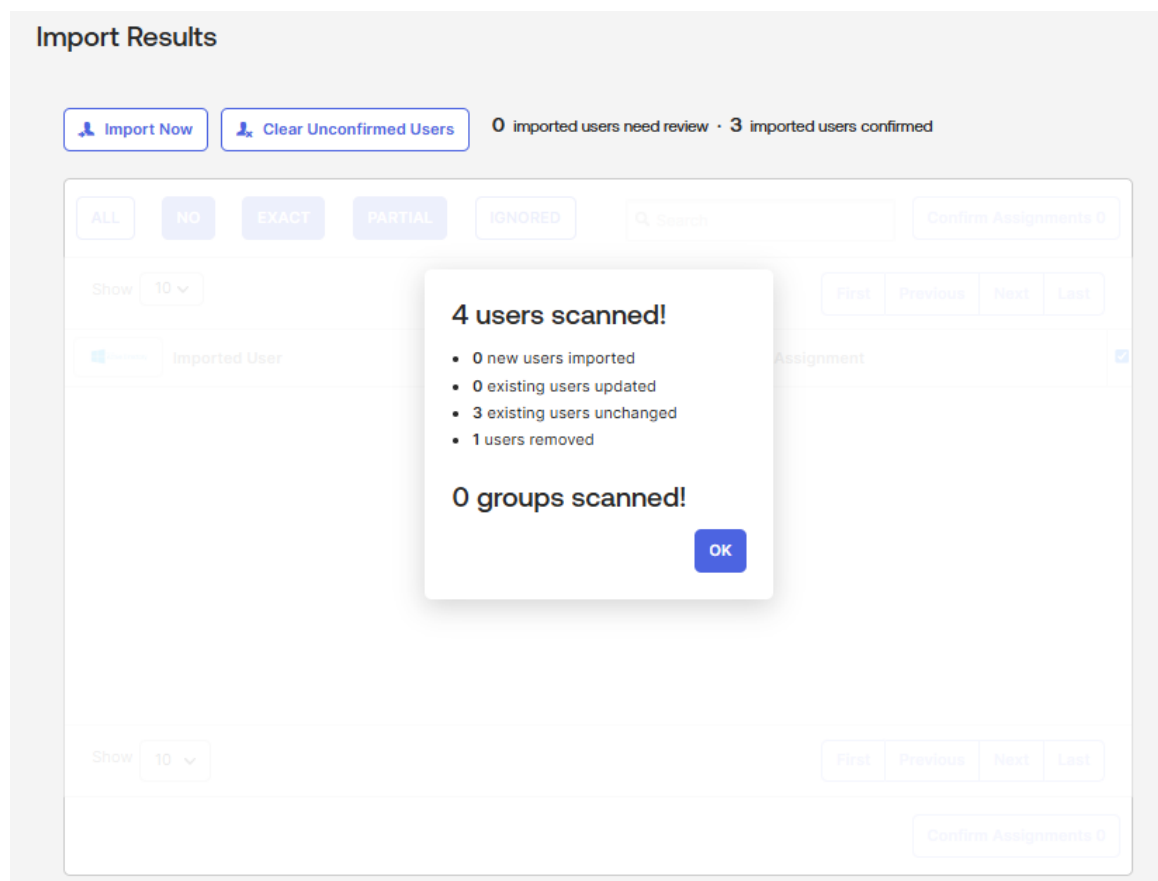
[*] Initiating automated offboarding protocol for: admin.test@local.Administrator.com
[+] Active Directory Response:
SUCCESS: Account admin.test@local.Administrator.com has been disabled and locked down.

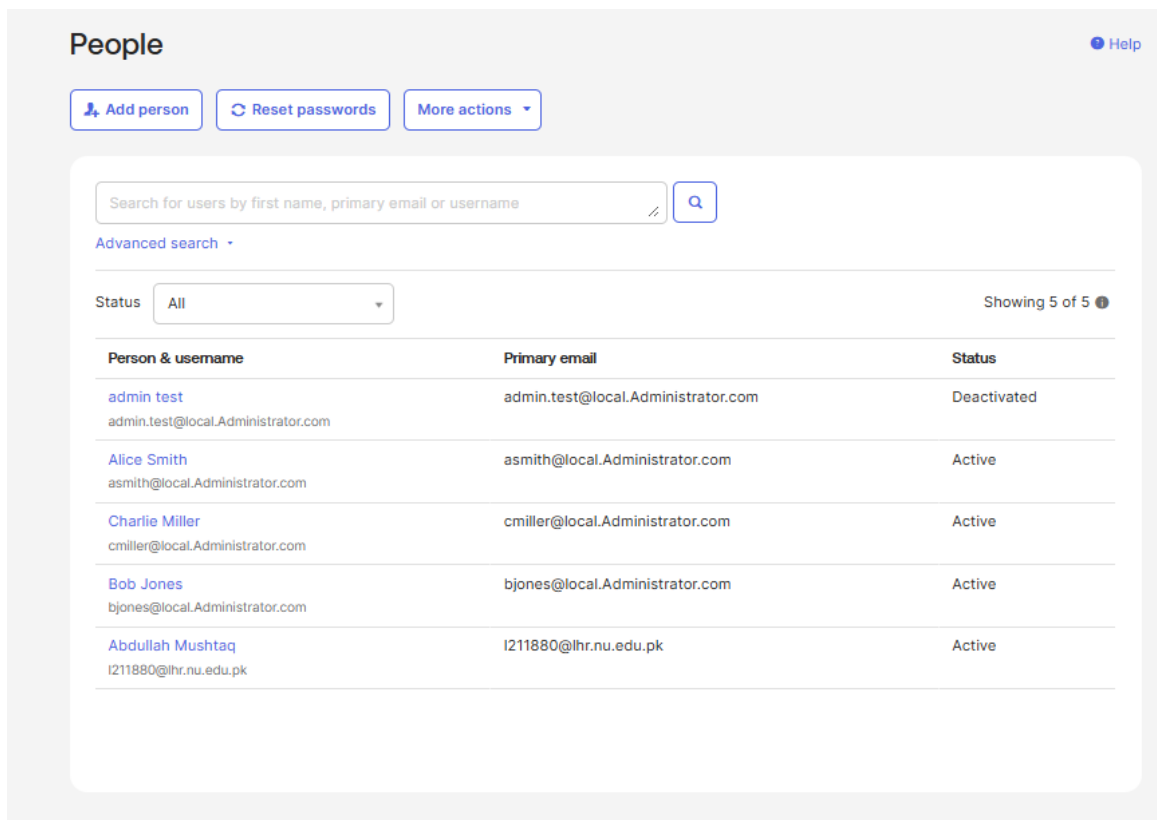
```

Open **Active Directory Users and Computers**, go to your **Enterprise_Users** OU, and look at the admin test account. You will see a little downward-pointing arrow on the account icon, meaning it is completely disabled. If you check the user's properties, you'll also see the automated timestamp in the description.



Next time Okta runs its sync (or if you manually click "Import Now" in Okta), that cloud account will automatically be suspended because the local AD account was disabled.





Conclusion

This laboratory exercise successfully demonstrated the design and implementation of a robust, hybrid Identity and Access Management (IAM) architecture. By bridging a custom-built, on-premise Active Directory environment with the Okta cloud platform, the project established seamless and centralized identity federation. Overcoming native cryptographic protocol mismatches and strict access controls ensured a highly secure, outbound communication tunnel from the local domain to the cloud infrastructure.

Beyond fundamental directory synchronization, this deployment heavily emphasized modern incident response readiness. The successful execution of a Python-driven, zero-touch de-provisioning pipeline via a secure WinRM bridge illustrates how programmatic automation can completely eliminate human latency during critical security events. Furthermore, by hardening the active domain with strict account lockout policies and comprehensive identity auditing, the architecture is fully primed for advanced SOC operations, including direct SIEM telemetry integration and automated SOAR workflows.

Ultimately, this project effectively bridges the gap between traditional systems administration and modern security automation. It serves as a functional proof-of-concept that a continuously monitored and fully automated identity lifecycle is the most critical perimeter for securing a scalable enterprise network.